- Bob browses a message board and views a post from a hacker where there is a crafted HTML image element. The element references a command in Bob's project management application, rather than an image file.
- `<img src="http://www.webapp.com/project/1/destroy">`
- Bob's session at www.webapp.com is still alive, because he didn't log out a few minutes ago.
- By viewing the post, the browser finds an image tag. It tries to load the suspected image from www.webapp.com. As explained before, it will also send along the cookie with the valid session id.
- The web application at www.webapp.com verifies the user information in the corresponding session hash and destroys the project with the ID 1. It then returns a result page which is an unexpected result for the browser, so it will not display the image.
- Bob doesn't notice the attack - but a few days later he finds out that project number one is gone.

It is important to notice that the actual crafted image or link doesn't necessarily have to be situated in the web application's domain, it can be anywhere - in a forum, blog post or email.

CSRF appears very rarely in CVE (Common Vulnerabilities and Exposures) - less than 0.1% in 2006 - but it really is a 'sleeping giant' [Grossman]. This is in stark contrast to the results in many security contract works - *CSRF is an important security issue*.

3.1 CSRF Countermeasures

First, as is required by the W3C, use GET and POST appropriately. Secondly, a security token in non-GET requests will protect your application from CSRF.

The HTTP protocol basically provides two main types of requests - GET and POST (and more, but they are not supported by most browsers). The World Wide Web Consortium (W3C) provides a checklist for choosing HTTP GET or POST:

Use GET if:

- The interaction is more *like a question* (i.e., it is a safe operation such as a query, read operation, or lookup).

Use POST if:

- The interaction is more *like an order*, or
- The interaction *changes the state* of the resource in a way that the user would perceive (e.g., a subscription to a service), or
- The user is *held accountable for the results* of the interaction.

If your web application is RESTful, you might be used to additional HTTP verbs, such as PATCH, PUT or DELETE. Most of today's web browsers, however do not support them - only GET and POST. Rails uses a hidden `_method` field to handle this barrier.

POST requests can be sent automatically, too. Here is an example for a link which displayswww.harmless.com as destination in the browser's status bar. In fact it dynamically creates a new form that sends a POST request.

```
<a href="http://www.harmless.com/" onclick="
  var f = document.createElement('form');
  f.style.display = 'none';
  this.parentNode.appendChild(f);
  f.method = 'POST';
  f.action = 'http://www.example.com/account/destroy';
  f.submit();
  return false;">To the harmless survey</a>
```

Or the attacker places the code into the onmouseover event handler of an image: